

auto-aiming

Revitalized auto-aiming suite for Purdue RoboMaster Club 2024-2025.

 Run Colcon Tests passing

Brief Overview of the Suite

Auto-Aim

Our auto-aiming system uses an industrial camera to detect enemy robots and launch shots at them accurately. Built in ROS2, the system is modular, with each step in the pipeline handled by separate nodes. First, it uses traditional computer vision techniques to detect an enemy robot's armor plate, and uses `solvepnp` to estimate its 3D position relative to our camera. A ballistics solver then calculates the yaw and pitch angles needed to hit the target. Basic filtering is applied to reduce false positives, and the final aiming commands are sent to the robot's control board over UART.

Navigation

Our navigation system enables the Sentry robot to autonomously move around the RoboMaster field, localize itself, and reach any spot on the map. Built using ROS2's Nav2 framework, it combines LiDAR data and wheel odometry to localize via a method called AMCL. A custom behavior script `subscriber.py` controls where and when the robot moves, while the path planning plugin (DWB) computes obstacle-avoiding paths and issues movement commands.

For more details on workings, issues, and possible improvements, see the [State of the Algorithm](#)

Installation and Usage

Clone this repository into your `~/ros2-ws` directory. We provide a `run` script that can be used to build, run, test, and clean the workspace with no hassle. The script supports the following functionalities:

- **Building**
 - `./auto-aiming/run build`
- **Run ROS2 code**
 - `./auto-aiming/run run <launch_file>`
 - "launch files" are how you start up the pipeline. For example you may use `video2detector.py` to run the auto-aiming pipeline with a video file, or `mv2control.py` to run using a real camera and send results to the STM32 control board.
- **Run automated tests (GTest)**
 - `./auto-aiming/run test`
- **Clean the workspace (remove build and install folders)**
 - `./auto-aiming/run clean`
- *Optional flags*

- `--quiet`: Suppresses console output, logs output to `command_output.log`.
- `--debug`: Builds with debug flags enabled. When enabled, displays a detection results window and debug logs.

Example to run the detector:

```
./auto-aiming/run --debug --quiet run video2detector.py
```

To run the navigation stack, please see the README in the `src/prm_autobot_2023` directory.

Overall Suite Requirements

Functional Requirements:

- ☒ **Detect an enemy armor plate in the camera's FOV.**
 - ☐ Meet the following detection rate and accuracy requirements:
 - ☐ 5 meters: 90% detection rate, 5% pixel loss
 - ☒ 3 meters: 95% detection rate, 5% pixel loss
 - ☒ 2 meters: 95% detection rate, 5% pixel loss
 - ☒ Reduce search area around previously detected plates ("search area reduction").
 - ☒ Achieve 120 Hz detection frequency.
 - ☐ Classify the robot type based on its armor plate sticker.
- ☒ **Compute camera-relative XYZ pose via PnP solving with 5% error margin.**
- ☐ **Filter out false positives and noise in the detection results.**
 - ☐ Use a Kalman filter to smooth XYZ pose estimates.
 - ☒ Apply a "validity filter" for erroneous detection/pose results (e.g., based on distance, XYZ shifts).
- ☒ **Compute the gimbal angles (yaw and pitch) required to accurately land projectiles on the detected armor plate.**
 - ☒ Compute pitch using an easily-adjustable lookup table or ballistic model based on distance to target.
 - ☐ Compute yaw using a predictive model using the detected armor's XYZ pose, rotation, and velocity
- ☒ **Send the computed gimbal angles to the STM32 control board via UART.**

Non-Functional Requirements:

- ☒ **Performance**
 - Ensure real-time end-to-end performance of 120 Hz.
- ☐ **Testability**
 - Include a comprehensive suite of unit tests for all modules to verify component correctness.
- ☒ **Maintainability**

- Maintain modularity by separating ROS2 and C++ logic into `xyzNode.cpp` and `xyz.cpp` files.
- Provide thorough documentation, including doxygen-style comments for functions and README files for modules.

Architecture Diagram

